

B28: SQL SuperHero Program : Session 6

1 message

Vishal Jaiswal <vishaldbdev@gmail.com>
Bcc: satishhattkar1997@gmail.com

18 February 2025 at 21:08

Modify Table:

--> we can modify the type size of any column using the alter table modify command, we can increase it easily but can not decrease it if any data exists because it will check the existing data and if the size is greater than that then deny to decrease.

```
create table employee (empid number, ename varchar2(20), age number, sex char(1))
```

Table created.

→ Table created

```
desc employee
```

TABLE EMPLOYEE

Column	Null?	Type
EMPID	-	NUMBER
ENAME	-	VARCHAR2(20)
AGE	-	NUMBER
SEX	-	CHAR(1)

4 rows selected.

*→ size of column
→ maximum 20
defined instat
(Means, we cannot insert anything
more than 20)*

```
insert into employee values (1, 'aJGF,AJgdv<sgjdMSJGHDKJSDGH', 34, 'm')
```

if gives error

ORA-12899: value too large for column "SQL_RUGBYIEQDKXLFJCQZYBBYGKRB"."EMPLOYEE"."ENAME" (actual: 27, maximum: 20)

More Details: <https://docs.oracle.com/error-help/db/ora-12899>

If we see, we can get errors that maximum 20 defined and you can not insert more than that. we can alter the table and increase or decrease the size.

```
alter table employee modify ename varchar2(30)
```

Table altered.

command

```
desc employee
```

TABLE EMPLOYEE

Column	Null?	Type
EMPID	-	NUMBER
ENAME	-	VARCHAR2(30)
AGE	-	NUMBER
SEX	-	CHAR(1)

→ size increased

4 rows selected.

```
alter table employee modify ename varchar2(28)
```

Table altered.

we can decrease as well.

```
desc employee
```

TABLE EMPLOYEE

Column	Null?	Type
EMPID	-	NUMBER
ENAME	-	VARCHAR2(28)
AGE	-	NUMBER
SEX	-	CHAR(1)

→ it was 30, now decrease to 28.

4 rows selected.

--> Increasing the length is easy, straight forward, but in case of size reduction, it will check each and every record of the database. It will be a slow and time taking process. Advisable in downtime only. See the below example, when we try to decrease it go and check each record in the column.

```
alter table employee modify ename varchar2(25)
```

ORA-01441: cannot decrease column length because some value is too big

More Details: <https://docs.oracle.com/error-help/db/ora-01441>

→ it will check each record.

Add Column:

--> add a column in table

alter table <table name> add <column nname> <data type>;

```
alter table employee add address varchar2(100);
```

if the table already has records, then if we add a new column then what happens.
The new column comes with a NULL value for old records.

The screenshot shows a SQL Developer window with the following SQL statement executed:

```
3 alter table employee add address varchar2(100);
```

Below the statement, the 'Query Result' tab displays the table structure and data. The table has columns EMPID, EMPNAME, AGE, SEX, and ADDRESS. The ADDRESS column is highlighted in blue, and all its values are (null). A handwritten note in blue ink says 'New Column add with null values' with an arrow pointing to the ADDRESS column.

EMPID	EMPNAME	AGE	SEX	ADDRESS
1	1 hsgsggs	100	M	(null)
2	2 yeyyete	50	F	(null)
3	3 5	30	M	(null)
4	4 75757245kjgdjgsfljfglf	23	F	(null)
5	1 bsgstye	34	F	(null)
6	1 bsgstye	34	(null)	(null)

if we want to add a new column and that column can not be null ?

alter table <tablename> add <column name> <datatype> default <value>;

```
alter table employee add address_type varchar2(20) Default 'Residential';
```

```
alter table employee add address_type varchar2(20) default 'residential'
```

Table altered.

```
select * from employee
```



EMPID	EMPNAME	AGE	SEX	ADDRESS	ADDRESS_TYPE
12	geugfe	34	F	-	residential
12	vish	35	F	-	residential
1	vish	36	M	-	residential
16	sjakgfkjaefejgkyfioegfefej	45	M	-	residential
12	vishal	35	M	-	residential
15	sjakgfkjaefejgfej	45	M	-	residential

Automatic
add

If you want to add more than one column, you can list them out in the add clause:

```
alter table <tablename> add (  
    column1    datatype,  
    column2    datatype  
);
```

DROP COLUMN

```
alter table <tablename> drop column <column name >
```

```
alter table employee drop column ADDRESS;
```

/* Advance concepts Good for 6+ years exp */

But beware!

This is an expensive operation. On tables storing millions of rows it will take a long time to run. And it blocks other changes to the table. So your application may be unusable while this runs!

A better option is to set the column unused:

```
alter table <tablename> set unused column <columnname>;
```

This is an instant operation. No matter how many rows are in the table, it will take the same amount of time.

This is because it doesn't physically remove the columns from the database. It marks them as unavailable. The data still exists. You just can't get to it!

If you're hoping to reclaim the space these columns used, you need to wipe them from the table. Do this with the following command:

```
ALTER TABLE <tablename> DROP UNUSED COLUMNS;
```

You can then query the dba_unused_col_tabs view which displays a list of all tables with un-used columns, including counts of the number of columns within a table that are unused.

```
select * from dba_unused_col_tabs;
```

Once you have decided to physically drop the column you can issue this DDL:

```
alter table mytab drop unused columns;
```

As you're now doing the work of deleting the data, this can take a long time. The key difference here is dropping unused columns is non-blocking. Your application can continue to run as normal.

Whichever method you use, take care when removing columns. Dropping columns or setting them unused are both one-way operations. There is no "undrop column" command. If you made a mistake, you'll have to recover the table from a backup!

--> SQL Server doesn't have a mechanism for marking columns as "unused," which is an Oracle extension of SQL.

Invisible column

An invisible column is a new concept in Oracle 12c. It's a column in a table. That does not appear in SELECT * queries. So, the only way to see the data inside it is to specify the column name in the SELECT clause.

Why Would You Use an Invisible Column?

In short, to improve security.

One of the benefits of using views in Oracle is to hide data from users who aren't allowed to see it. You can specify a view to only select some of the columns of a table, and give users access to the view and not the underlying table. Then, according to the users, the other columns don't exist. An invisible column works in much the same way.

you can set a column in the table as invisible either during CREATE TABLE or modifying existing table via ALTER TABLE command.

```
create table test (  
  col1 number,  
  col2 number,  
  col3 number invisible  
)
```

→ Column is invisible

Table created.

```
insert into test (col1,col2) values (1,2)
```

1 row(s) inserted.

← inserted for other column

```
select * from test
```

COL1	COL2
1	2

→ Not appear

```
select col1,col2,col3 from test
```

COL1	COL2	COL3
1	2	-

Specify then
I can see

```
insert into test values (1,2)
```

1 row(s) inserted.

No issue here
too

How we can see if that table has a hidden column. Go to Data dictionary User_tab_cols

```
select column_id, column_name, hidden_column from user_tab_cols where table_name = 'TEST'
```

COLUMN_ID	COLUMN_NAME	HIDDEN_COLUMN
1	COL1	NO
2	COL2	NO
-	COL3	YES

3 rows selected.

Yes
No id assigned

we learned on Invisible Column, lets continue here with one scenario based questions:

In Oracle, Can we add a new column in a table in a specific position, without dropping the table. My table ABC, has column Id number, name varchar2(20). I want to add a new salary column, but this should be 2nd column in the table. How to achieve this?

```
create table A (id number, name varchar2(20))
```

Table created.

```
alter table A add email varchar2(50)
```

Table altered.

→ New column added

```
desc a
```

TABLE A

Column	Null?	Type
ID	-	NUMBER
NAME	-	VARCHAR2(20)
EMAIL	-	VARCHAR2(50)

→ Added in last

3 rows selected.

```
alter table A modify name invisible
```

Table altered.

→ Make the 2nd one

```
desc a
```

TABLE A

Column	Null?	Type
ID	-	NUMBER
EMAIL	-	VARCHAR2(50)
NAME	-	VARCHAR2(20)

→ This become last

3 rows selected.

```
alter table A modify name visible
```

Table altered.

